Assignment 1: Multi-Instance Object Recognition in Cluttered Scenes

Objective

Create a pipeline that detects and localizes multiple instances of a specific object within a highly cluttered and occluded scene. You need to consider scale and rotation invariance as well as robustness against extreme background noise. **For this assignment, you may use OpenCV for the initial SIFT feature extraction, but the matching, clustering, and geometric verification must be implemented entirely from scratch.**

1. Data Collection

a. Take a clear, isolated template image of a single common object from your hostel environment (e.g., a specific brand of instant noodles packet, a standard hostel padlock, a specific steel mess cup, or a uniquely patterned notebook).

b. Take a query image in a highly cluttered environment (like a messy hostel desk, a crowded cycle stand, or a tree outside your window with multiple birds if using a bird template) containing **at least three** instances of your chosen object.

c. Make sure the objects in the query image present a challenge: they should be partially occluded (hidden behind other things), rotated at odd angles, and placed at different distances (scales) from the camera.

2. Recognition Pipeline

a. You are allowed to use standard libraries (such as OpenCV, NumPy, Matplotlib) for image I/O, matrix operations, and extracting SIFT features (`cv2.SIFT_create`). However, you are **NOT allowed** to use high-level automated matching, clustering, or geometric APIs (such as `cv2.BFMatcher`, `cv2.FlannBasedMatcher`, `cv2.estimateAffine2D`, `cv2.findHomography`, or `sklearn.cluster`). The core mathematical logic for finding the objects must be implemented from scratch. **Note: No boilerplate code is provided for this assignment. You are responsible for structuring the pipeline yourself.**

b. The following topics should be addressed in your implementation:

- **SIFT Feature Extraction:** Use OpenCV to detect keypoints and compute the 128-dimensional local image descriptors for both the template and the cluttered scene.
- **Feature Correspondence (From Scratch):** Find and match points of interest between your template and the cluttered scene. Compute the Euclidean distances between descriptors yourself. You must implement Lowe's relative distance ratio check to reject ambiguous matches that look too similar to background noise. *(Hint: Match features from the Scene to the Template to avoid failing on multiple instances).*

- **Parameter Space Clustering (From Scratch):** Standard robust estimation fails when the object is heavily occluded because the ratio of correct matches to background noise is extremely low. Instead, implement a 4D Generalized Hough Transform. For every matched feature, calculate the predicted 2D center location, scale, and orientation of the object. Group these predictions into discrete bins to find highly probable object locations.
- **Robust Geometric Transformation (From Scratch):** Once a valid cluster of matches is found (e.g., a bin with ≥ 3 votes), use the matched points to find the coefficients of a 2D Affine transformation matrix. You must implement a robust iterative estimator (like RANSAC) combined with a Least-Squares solver ($A\mathbf{x} = \mathbf{b}$) to reject outliers within the cluster that cause the bounding box to shear. Implement geometric sanity checks (collinearity and determinant/scale checks) to prevent degenerate solutions.
- **Iterative Processing:** Repeat the localization and verification steps to find *all* valid instances of the object in your set. Use sequential inlier subtraction (Greedy Extraction Loop) to process each valid cluster in your Hough space to build the final detected outputs.

3. Output Refinement & Visualization

A "raw" detection pipeline can cause messy visual outputs with overlapping predictions or severe shearing. You are supposed to:

- **Draw Boundaries:** Project the corners of your template image into the scene using your calculated affine transformation coefficients. The boundaries should neatly encapsulate the target objects without heavy distortion.
- **Suppress Duplicates:** Make sure that multiple detections of the exact same object instance are merged or filtered out using Intersection over Union (IoU) Non-Maximum Suppression (NMS) so only one clean bounding box remains per actual object.
- **Deal with False Positives:** Implement an error-threshold verification check to ensure background hostel clutter isn't accidentally boxed as an object.
- **Final Cleanup:** The final image should be clean, highlighting the detected objects with clear bounding boxes without overwhelming the image with stray keypoint markers or messy matching lines.

Deliverables

- **Original Photos:** The isolated template photo and the cluttered query photo.
- **Codebase:** Clean, well-documented code that clearly explains the logic step by step.
- **Report:** Show a comparison between a "naive" match (direct feature matching overlay with all its chaotic background noise) and your final processed bounding box detections in a small report while describing your mathematical implementation.